Stack vs Heap: How Java Organises Memory

12 min read · Updated 28 Jul 2026 · Topic 4 of 65

Why it matters

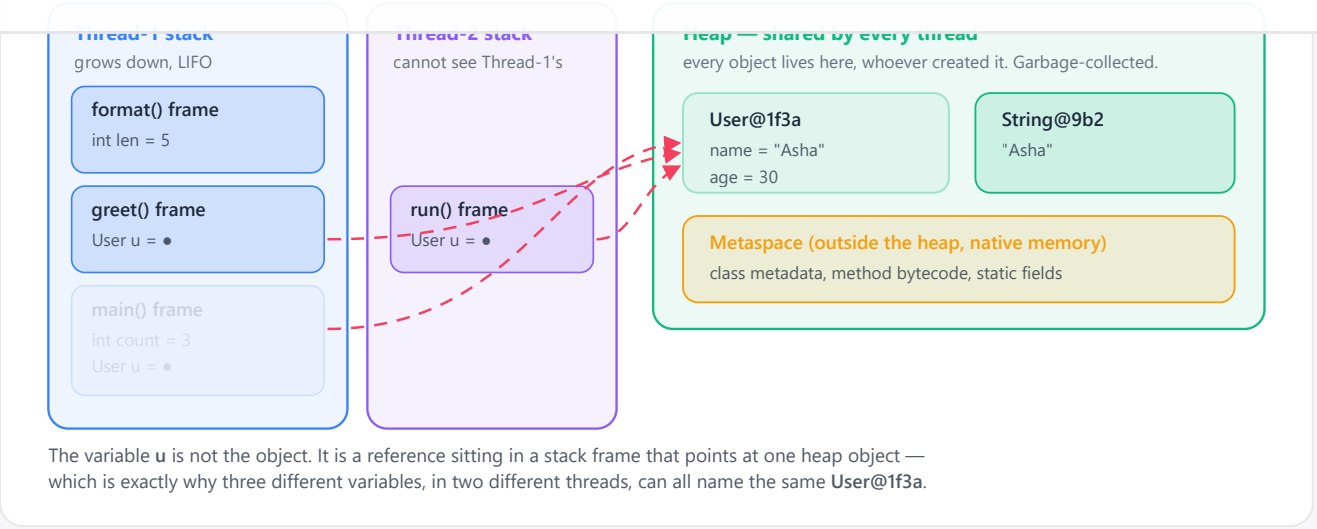
Ask any Java interviewer for their favourite diagnostic question and "stack versus heap" will be near the top, because the answer predicts so much else. A candidate who holds this model correctly can reason about why one thread cannot corrupt another's locals, why a recursive method blows up with a different error than a memory leak does, what a thread dump is showing them, and why `-Xmx` and `-Xss` are separate knobs. A candidate who does not is reduced to memorising each of those facts separately.

It is also the model you will actually use in production. Every `OutOfMemoryError` and `StackOverflowError` you ever debug starts with knowing which region ran out and why.

The concept, simply

The JVM divides memory into regions with very different rules. Two matter most:

- The **stack** is per-thread and private. Each thread gets its own, and it holds one *frame* per method call in progress: that method's local variables, its parameters, and where to return to. Frames are pushed on call and popped on return, strictly last-in-first-out. Memory management is therefore free — returning from a method reclaims its frame instantly, with no garbage collector involved.
- The **heap** is shared by every thread and holds *every object*. It is not ordered, objects live as long as something still references them, and reclaiming them is the garbage collector's job.



Stacks are private and orderly; the heap is shared and unordered. The arrows are the whole point — a local variable is a reference in a frame, and several frames across several threads can point at one object.

A third region completes the picture: **Metaspace**, which holds class metadata — the loaded classes themselves, their method bytecode, and their static fields. It lives in *native* memory, outside the heap, and since Java 8 it replaced the old fixed-size PermGen. That change is why `java.lang.OutOfMemoryError: PermGen space` is a message you will only see in legacy systems.

How it works

The stack, precisely. Each thread receives its own stack when it starts, with a size fixed by `-Xss` (commonly around 512 KB to 1 MB). Every method invocation pushes a frame holding the local variable array, the operand stack the bytecode manipulates, and a reference to the constant pool. When the method returns, the frame pops and everything in it disappears. Because the stack is private to one thread, **locals are automatically thread-safe** — no other thread can even name them. That is a fact worth carrying into Chapter 8, where it becomes the cheapest concurrency strategy there is.

The heap, precisely. Every object created with `new` — and every array, and every boxed value — is allocated here. Its size is bounded by `-Xms` (initial) and `-Xmx` (maximum). Objects survive as long as they are *reachable* from a GC root; when they are not, they become eligible for collection. The heap is further divided into generations (young, old) because most objects die young — Chapter 9 covers why that observation shapes the entire GC design.

Thing	Lives in
A primitive *local* variable (`int i` in a method)	Stack — inside the frame
A reference *local* variable (`User u` in a method)	Stack — the reference. The object is on the heap.
A primitive *field* (`int age` in a class)	Heap — inside the object
Any object, any array	Heap
`static` fields	Metaspace (the objects they point at are on the heap)
Class metadata, method bytecode	Metaspace
The string pool	Heap (moved out of PermGen in Java 7)

The row people miss is the third: a primitive field is *not* on the stack. `int age` declared inside a class lives inside that object, on the heap, for the object's whole life. Only *locals* live on the stack. The rule is not "primitives on the stack, objects on the heap" — it is "**locals on the stack, objects (and their fields) on the heap**".

Code examples

Trace one method and say exactly where each piece lives:

JAVA

Copy

```java
public class Memory {
    static int counter = 0;              // METASPACE (static field)
    int instanceField = 7;               // HEAP — inside each Memory obje

    public static void main(String[] args) {
        int localPrimitive = 42;         // STACK — main's frame
        Memory m = new Memory();         // reference on STACK, object on
```

```
        m.doWork(localPrimitive);        // pushes a new frame
    }                                    // main returns → its frame pops

    void doWork(int input) {             // input: STACK — doWork's own fr
        int doubled = input * 2;         // STACK
        instanceField = doubled;         // writes into the HEAP object
    }                                    // frame pops; input and doubled
}
```

The two errors, and why they are genuinely different:

JAVA Copy

```
// StackOverflowError — the STACK ran out. Too many frames, usually
// runaway recursion. Nothing to do with how much heap you have.
static int depth(int n) {
    return depth(n + 1);        // no base case
}
// Exception in thread "main" java.lang.StackOverflowError

// OutOfMemoryError: Java heap space — the HEAP ran out. Too many live
// objects. Raising -Xss would not help by a single byte.
List<byte[]> leak = new ArrayList<>();
while (true) {
    leak.add(new byte[10 * 1024 * 1024]);   // 10 MB, and kept reachable
}
// Exception in thread "main" java.lang.OutOfMemoryError: Java heap space
```

Stack privacy is what makes local state thread-safe for free:

JAVA Copy

```
public class Counting implements Runnable {
    public void run() {
        int local = 0;                  // one INDEPENDENT copy per thread
        for (int i = 0; i < 1000; i++) local++;
        System.out.println(local);      // always 1000, from every thread
    }
```

```
// Change "local" to a shared field and you have a race condition (Chapter
```

The knobs, for completeness:

BASH Copy

```
java -Xms512m -Xmx2g -Xss1m Main
#      |        |        └─ stack size PER THREAD (more threads = more tota
#      |        └────────── max heap
#      └─────────────────── initial heap (set == -Xmx in servers to avoid

# Dump the heap automatically when it blows up — the single most useful
# production flag in this area.
java -XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/var/log/app.hprof M
```

Under the hood

Allocation on the heap is faster than people assume. The intuitive model — search a free list for a gap big enough — is not what a modern JVM does. Generational collectors keep the young generation compacted, so allocation is usually a *pointer bump*: take the current top-of-space pointer, add the object's size, done. On top of that each thread gets a **TLAB** (Thread-Local Allocation Buffer), its own private slice of Eden, so the bump needs no synchronisation at all. Allocation is therefore a handful of instructions, and "object creation is expensive in Java" is folklore inherited from the 1990s.

Stack size interacts with thread count. `-Xss` is per thread, so a server running 2,000 threads at 1 MB each has reserved 2 GB of stack — entirely outside the heap, and invisible to `-Xmx`. This is a classic cause of "the container is OOM-killed but the heap looks fine": the kernel counts stack memory, and your heap graph never showed it. It is also part of the motivation for virtual threads in Java 21 (Chapter 10), whose stacks live on the heap and grow on demand rather than reserving a fixed slab up front.

The escape-analysis caveat, again. "All objects live on the heap" is the model to state, and it is right for reasoning about your program. Physically, the JIT may prove an object

pedantic.

Common mistakes

- **"Primitives live on the stack."** Only primitive *locals* do. A primitive field lives inside its object, on the heap.
- **Confusing the two errors.** `StackOverflowError` = too deep, tune `-Xss` or fix the recursion. `OutOfMemoryError: Java heap space` = too many live objects, tune `-Xmx` or fix the leak. Neither flag helps the other problem.
- **Thinking a memory leak is impossible with a GC.** The GC frees *unreachable* objects. An object you forgot to remove from a static `Map` is perfectly reachable, and will never be freed. That is a leak.
- **Believing static fields are on the stack.** They live in Metaspace, and outlive every method call.
- **Sizing only the heap in a container.** Stacks, Metaspace, code cache and direct buffers are all outside `-Xmx`. Container limits must account for them or the process gets killed.
- **Reaching for `-Xss` to fix deep recursion.** It buys a constant factor. If the recursion is unbounded, no stack size is enough — restructure it iteratively.

Interview lens

> **Q: "Explain stack versus heap in Java."**
>
> A: The stack is per-thread and private; it holds a frame per in-progress method call containing that method's locals and parameters, pushed and popped LIFO, so memory is reclaimed automatically on return with no GC involvement. The heap is shared across all threads and holds every object; objects live while they remain reachable and are reclaimed by the garbage collector. A local reference variable sits on the stack, but the object it points at is on the heap.

A: On the heap, inside the object that declares it. Only *local* variables live on the stack. The rule is locals-on-the-stack, objects-and-their-fields-on-the-heap — not primitives-versus-objects.

Q: "Difference between `StackOverflowError` and `OutOfMemoryError`?"

A: Different regions. `StackOverflowError` means one thread pushed more frames than its stack can hold — almost always unbounded recursion. `OutOfMemoryError: Java heap space` means the heap cannot satisfy an allocation because too much is still reachable — either a genuine leak or an undersized heap. Both are `Error`s rather than exceptions, so you do not catch them in normal code.

Q: "Why are local variables thread-safe?"

A: Because each thread has its own stack. A local variable exists only inside one thread's frame, so no other thread can reach it — there is nothing to synchronise. The caveat is that the local is only safe if it is a primitive or a reference to an object no one else shares; if it points at a shared object, that object still needs protecting.

Q: "Where do static variables live?"

A: In Metaspace, with the class metadata — native memory, outside the heap, since Java 8 replaced PermGen. If a static field holds a reference, the referenced object is still on the heap; the static field is a GC root, which is exactly why a static collection that is never cleared is such a reliable memory leak.

Q: "Can Java have a memory leak?"

A: Yes. The GC collects unreachable objects, not useless ones. Objects added to a long-lived static collection and never removed, listeners never unregistered, `ThreadLocal`s never cleaned up in a pooled thread — all stay reachable, so all

stay alive. That is a leak by any practical definition, and it is diagnosed with a heap dump rather than by reading code.

> **Q: "You get an OOM in production. What do you do?"**
> A: First read which OOM it is — heap space, Metaspace, GC overhead limit, and "unable to create native thread" have completely different causes. Assuming heap space: run with `-XX:+HeapDumpOnOutOfMemoryError` so I have a dump, then open it in Eclipse MAT and look at the dominator tree for what is retaining the most memory and which GC root holds it. That usually names the leak in minutes. Raising `-Xmx` is a stopgap that only buys time if the growth is unbounded.

Cheat-card

- **Stack** = per thread, private, frames, LIFO, auto-reclaimed. `-Xss`.
- **Heap** = shared, all objects, GC-managed. `-Xms` / `-Xmx`.
- **Metaspace** = class metadata + statics, native memory, replaced PermGen in Java 8.
- Rule: **locals on the stack; objects and their fields on the heap**.
- Local reference on the stack → object on the heap. The arrow crosses regions.
- `StackOverflowError` = too deep. `OutOfMemoryError` = too much live data. Different knobs.
- Locals are **thread-safe for free** — nothing else can reach them.
- Allocation is a **pointer bump in a TLAB**, not a free-list search. Cheap.
- Leaks happen: reachable ≠ useful. Static maps, listeners, `ThreadLocal`s.
- Container sizing must count stacks + Metaspace + code cache, all **outside** `-Xmx`.

QUICK CHECK — no marks, just for you.

1. Where does the field `int age;` declared inside a class live?

- ◯ On the heap, inside the object that declares it

On the stack, in the frame of whichever method reads it

- In Metaspace, alongside the class metadata
- In the constant pool of the .class file

Check answer

2. A method recurses without a base case. Which error do you get, and which JVM flag is relevant?

- OutOfMemoryError: Java heap space — raise -Xmx
- StackOverflowError — the stack is sized by -Xss, though the real fix is bounding the recursion
- OutOfMemoryError: Metaspace — raise -XX:MaxMetaspaceSize
- StackOverflowError — the stack is sized by -Xmx

Check answer

3. Why are local variables thread-safe without any synchronisation?

- The JVM automatically synchronises access to method-local state
- They are immutable once assigned
- Each thread has its own stack, so a local exists only in that thread's frame and no other thread can reach it
- The JIT compiler copies them into registers, which are per-core

Check answer

Previous
Primitives vs References

Next
Pass-by-Value: Java's Most Misunderstood Rule

LEARN MORE

ARTICLES & DOCS

- Run-time data areas (JVMS §2.5)
  docs.oracle.com
- java — the launcher, and every -X option
  docs.oracle.com
- Eclipse MAT — the heap dump analyzer
  eclipse.dev

VIDEOS



Heap Space vs. Stack Memory in Java | Call Stack Explained | Memory Allocation | Geekific
Geekific



Java 8 JVM Architecture Explained 🔥 | Heap, Stack, Metaspace (PermGen?) & JVM Memory | Part 2
Vaibhav Code Lab

Hand-picked references for this lesson.